

Process Scheduling

Shaon Khan

Operating Systems — Chapter 05
Institute of Information Technology (IIT)
Jahangirnagar University

Table of Contents

Chapter 1	Introduction to Processes and Operating Systems
Chapter 2	Process Scheduling — Basics and Concepts
Chapter 3	CPU–I/O Burst Cycle
Chapter 4	Scheduling Criteria
Chapter 5	CPU Scheduler and Dispatcher
Chapter 6	Preemptive vs Non-Preemptive Scheduling
Chapter 7	Scheduling Algorithms
7.1	First-Come, First-Served (FCFS)
7.2	Shortest-Job-First (SJF)
7.3	Priority Scheduling
7.4	Round Robin (RR)

Chapter 1

Introduction to Processes and Operating Systems

1.1 What is a Process?

A process is a program in execution. When you double-click an application on your computer, the operating system creates a process for it. Think of a process as a living, running version of a program.

Consider a simple analogy:

- A recipe is like a program — it is just instructions written on paper.
- When a chef actually cooks using that recipe, that is like a process — the recipe is being executed.
- Multiple chefs can cook from the same recipe at the same time. Similarly, multiple processes can run from the same program.

Term	Simple Meaning	Example
Program	A set of instructions stored on disk	A .exe or .py file
Process	A program that is currently running	A running browser or game
Thread	A smaller unit of execution inside a process	A tab in a browser

1.2 What is a CPU?

The CPU (Central Processing Unit) is the brain of the computer. It executes instructions one at a time (or a few at a time on modern processors). Every process needs the CPU to run.

- A CPU can only run one process at a time on a single core.
- Modern computers have multiple CPU cores, allowing them to run multiple processes simultaneously.
- Because many processes compete for the CPU, there must be a system to manage which process gets the CPU and when.

1.3 What is an Operating System?

The Operating System (OS) is the master software that manages all hardware and software resources. It acts like a manager who distributes work to employees (processes).

Key OS responsibilities:

- Manage CPU time among all running processes.
- Manage memory (RAM) allocation.
- Handle input/output (I/O) devices like disk and network.
- Provide a user interface and security.

1.4 Why Do We Need Scheduling?

Imagine a bank with 100 customers but only 1 cashier. Without a rule for who gets served next, there would be chaos. The same problem exists in an operating system — many processes want the CPU, but only one can use it at a time.

Without scheduling:

- Some processes might wait forever.
- The CPU might sit idle even when there is work to do.
- Important tasks may not be completed on time.

With scheduling, the OS can:

- Keep the CPU busy as much as possible.
- Give every process a fair chance.
- Complete urgent tasks quickly.
- Respond quickly to user actions.



Chapter Summary

- A process is a program that is currently running.
- The CPU is the hardware that executes process instructions.
- The OS manages all processes and decides who gets the CPU.
- Scheduling is needed because many processes compete for one CPU.

Chapter 2

Process Scheduling — Basics and Concepts

2.1 What is Process Scheduling?

Process scheduling is the activity of the OS that decides which process gets the CPU at any given time. It handles:

- Removing the currently running process from the CPU.
- Selecting the next process to run, based on a specific strategy (algorithm).

Definition

Process scheduling is the mechanism by which the operating system manages the allocation of CPU time to processes, ensuring efficient utilization and fair resource distribution.

2.2 Multiprogramming

Multiprogramming means having multiple processes loaded into memory at the same time. The OS switches the CPU between them so that it is always doing useful work.

How it works:

1. Process P1 starts running on the CPU.
2. P1 needs to wait for disk I/O (reading a file). It cannot use the CPU during this wait.
3. Instead of letting the CPU sit idle, the OS gives the CPU to Process P2.
4. When P1's I/O finishes, it goes back to the ready queue and waits for its turn again.
5. This continues, keeping the CPU busy almost all the time.

Note: Without multiprogramming, the CPU would be idle whenever a process waits for I/O. Multiprogramming eliminates this waste.

2.3 Single-Core vs Multi-Core Systems

Feature	Single-Core CPU	Multi-Core CPU
Processes running at once	1 (others wait)	Multiple (one per core)
Scheduling need	Very important	Still important
Parallelism	None (simulated)	True parallelism
Example	Old computers	Modern laptops/phones



Chapter Summary

- Process scheduling decides which process runs on the CPU next.
- Multiprogramming keeps the CPU busy by switching processes.
- Even multi-core systems need scheduling because processes > cores.

Chapter 3

The CPU–I/O Burst Cycle

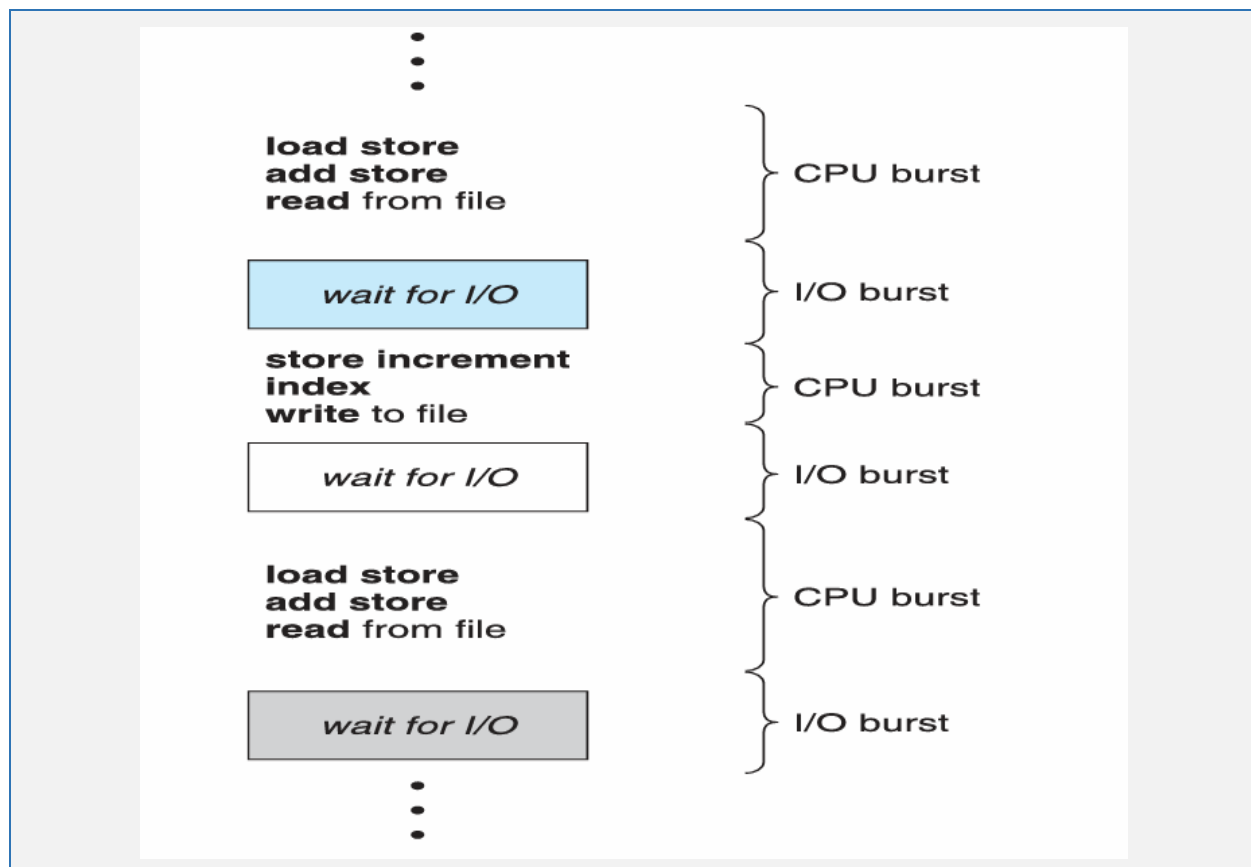
3.1 What is a Burst?

Every process alternates between two activities:

- CPU Burst — The process is actively using the CPU to perform calculations.
- I/O Burst — The process is waiting for an I/O operation (like reading from disk or network).

This alternation is called the CPU–I/O Burst Cycle. Every process goes through this cycle repeatedly until it terminates.

3.2 CPU–I/O Burst Cycle Diagram



The diagram shows a typical process execution flow:

1. CPU Burst: load, store, add, read from file
2. I/O Burst: wait for I/O
3. CPU Burst: store, increment, write to file
4. I/O Burst: wait for I/O
5. CPU Burst: load, store, add, read from file
6. I/O Burst: wait for I/O ... (continues until process ends)

3.3 CPU-Bound vs I/O-Bound Processes

Feature	CPU-Bound Process	I/O-Bound Process
Main activity	Heavy calculations	Frequent I/O operations
CPU burst duration	Long (few bursts)	Short (many bursts)
I/O burst duration	Short	Long
Examples	Video rendering, scientific computing	File copy, web browser, database
CPU usage	High	Low
I/O usage	Low	High

Note: Understanding whether a workload is CPU-bound or I/O-bound helps in choosing the right scheduling algorithm for best performance.

Chapter Summary

- A CPU burst is when the process uses the CPU actively.
- An I/O burst is when the process waits for I/O to complete.
- CPU-bound processes have long CPU bursts and few I/O waits.
- I/O-bound processes have short CPU bursts and many I/O waits.

Chapter 4

Scheduling Criteria

To evaluate and compare scheduling algorithms, we use the following five criteria. Each one measures a different aspect of performance.

Criterion	Definition	Goal	Formula
CPU Utilization	Percentage of time the CPU is actively working	Maximize (100% ideal)	$\text{CPU Busy Time} / \text{Total Time} \times 100\%$
Throughput	Number of processes completed per unit time	Maximize	$\# \text{ Processes Completed} / \text{Time}$
Turnaround Time	Total time from submission to completion of a process	Minimize	$\text{Completion Time} - \text{Arrival Time}$
Waiting Time	Total time a process spends waiting in the ready queue	Minimize	$\text{Turnaround Time} - \text{Burst Time}$
Response Time	Time from request submission until first response produced	Minimize (for interactive systems)	$\text{First Response Time} - \text{Arrival Time}$

4.1 Optimization Goals

A good scheduling algorithm aims to:

- Maximize CPU Utilization — keep the CPU as busy as possible.
- Maximize Throughput — complete as many processes as possible per second.
- Minimize Turnaround Time — finish each process quickly.
- Minimize Waiting Time — reduce the time processes spend idle in the queue.
- Minimize Response Time — respond quickly to user input (critical for interactive systems).

Note: In practice, optimizing all criteria simultaneously is impossible. Different algorithms make different trade-offs. For example, an algorithm that minimizes waiting time may increase overhead.

Key Formulas

Turnaround Time = Completion Time - Arrival Time

Waiting Time = Turnaround Time - Burst Time

Waiting Time = Start Time - Arrival Time (simple case)

Average Waiting = Sum of all Waiting Times ÷ Number of Processes

Chapter Summary

- CPU Utilization measures how busy the CPU is — we want it high.
- Throughput counts completed processes per second — we want it high.
- Turnaround Time is the total life of a process — we want it low.
- Waiting Time is idle time in the queue — we want it low.
- Response Time matters for interactive applications — we want it low.

Chapter 5

CPU Scheduler and Dispatcher

5.1 The CPU Scheduler

The CPU Scheduler (also called the Short-Term Scheduler) is the part of the OS that selects the next process from the ready queue and gives it the CPU.

- The ready queue contains all processes that are loaded in memory and waiting for CPU time.
- The queue may be ordered in various ways — FIFO, priority, shortest burst first, etc.
- The scheduler runs very frequently — sometimes every few milliseconds.

5.2 When Does Scheduling Happen?

CPU scheduling decisions are made in the following four situations:

Case	Event	Type
Case 1	A process switches from Running → Waiting state (e.g., waits for I/O)	Non-preemptive
Case 2	A process switches from Running → Ready state (e.g., time quantum expires)	Preemptive
Case 3	A process switches from Waiting → Ready state (e.g., I/O completes)	Preemptive
Case 4	A process terminates	Non-preemptive

Note: Cases 1 and 4 always occur even in non-preemptive systems. Cases 2 and 3 are only possible in preemptive scheduling.

5.3 The Dispatcher

The Dispatcher is a module that gives control of the CPU to the process selected by the scheduler. It is responsible for:

- Context Switching — saving the state of the old process and loading the state of the new process.
- Switching to User Mode — the OS runs in kernel mode; it must switch to user mode for the process.
- Jumping to the correct location — the dispatcher jumps to the exact instruction where the process should continue.

Dispatch Latency

Dispatch latency is the time it takes for the dispatcher to stop one process and start another. This is pure overhead — no useful work is done during this time. It should be as small as possible.

Chapter Summary

- The CPU Scheduler selects the next process from the ready queue.
- Scheduling happens in 4 cases: waiting, preempted, I/O done, terminated.
- The Dispatcher actually hands over the CPU to the selected process.
- Dispatch Latency is the overhead time during a context switch.

Chapter 6

Preemptive vs Non-Preemptive Scheduling

6.1 Non-Preemptive Scheduling

In non-preemptive scheduling, once a process gets the CPU, it keeps the CPU until it either finishes or voluntarily gives it up (by waiting for I/O).

- The CPU cannot be forcibly taken away from the process.
- Simpler to implement — no need to save state mid-execution.
- Scheduling only happens when a process terminates or waits for I/O (Cases 1 and 4).
- Response times are more predictable.
- Short processes may have to wait behind long ones — this can be unfair.

Real-life analogy: A cashier at a supermarket serves one customer completely before starting the next. Once a customer is being served, they cannot be interrupted.

6.2 Preemptive Scheduling

In preemptive scheduling, the OS can forcibly take the CPU away from a running process and give it to another process. This happens when:

- A higher-priority process arrives.
- The process has used its allotted time slice (time quantum).
- An I/O operation completes and a waiting process becomes ready.

Real-life analogy: A doctor's office where a patient with an emergency can interrupt a regular appointment. The regular patient waits while the emergency is handled.

6.3 Comparison Table

Feature	Non-Preemptive	Preemptive
CPU removal	Cannot be taken away mid-run	Can be taken away anytime
When it runs	Cases 1 and 4 only	All 4 cases
Response time	Predictable but slow	Fast, especially for high-priority tasks
Fairness	Short jobs may wait behind long ones	Better fairness with time slicing

Feature	Non-Preemptive	Preemptive
Complexity	Simpler to implement	More complex (context saving needed)
Overhead	Low	Higher (frequent context switches)
Examples	FCFS, Non-preemptive SJF	Round Robin, Preemptive Priority, SRTF

Chapter Summary

- Non-preemptive: process keeps CPU until it finishes or waits for I/O.
- Preemptive: OS can forcibly take the CPU from a running process.
- Preemptive offers better response time but has more overhead.
- Non-preemptive is simpler but less fair to short processes.

Chapter 7

Scheduling Algorithms

The following are the major CPU scheduling algorithms. Each has its own logic, advantages, and disadvantages. We will study each with step-by-step examples.

Algorithm	Type	Key Idea
FCFS	Non-preemptive	Serve processes in the order they arrive
SJF	Both	Serve the process with the shortest CPU burst first
Priority	Both	Serve the process with the highest priority first
Round Robin	Preemptive	Give each process a fixed time slice in rotation
Multilevel Queue	Both	Separate queues for different process types

7.1 First-Come, First-Served (FCFS)

FCFS is the simplest scheduling algorithm. Processes are served in the exact order they arrive — just like a queue at a ticket counter.

- Non-preemptive: once a process starts, it runs until completion.
- Easy to understand and implement.
- The ready queue is a simple FIFO (First In, First Out) queue.

Question 1 — FCFS Without Arrival Time

Question

Given:

- Three processes: P1, P2, P3
- Arrival time of all processes = 0
- Order of arrival: P1 → P2 → P3
- CPU burst times:
 - P1 = 24 ms
 - P2 = 3 ms
 - P3 = 3 ms

Using: First-Come, First-Served (FCFS) scheduling algorithm

Find:

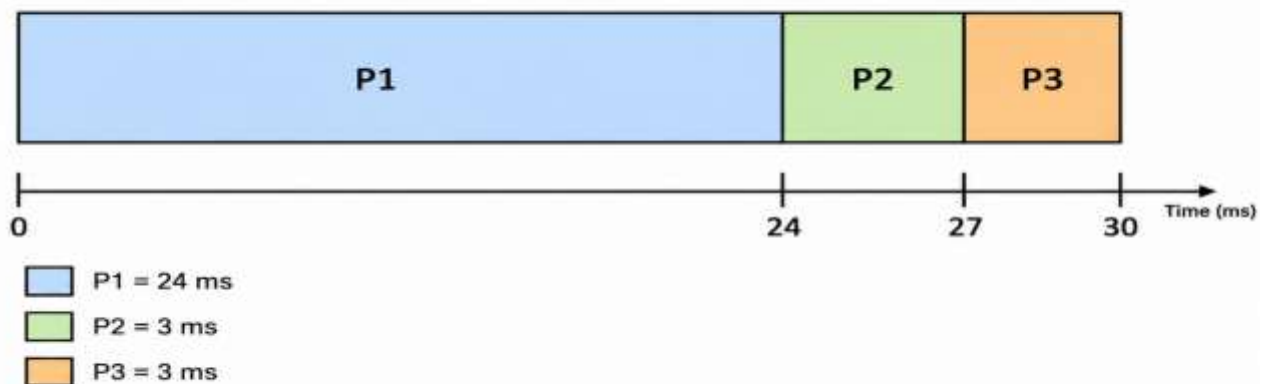
1. Draw the Gantt chart.
2. Calculate the waiting time for each process.
3. Calculate the average waiting time.

Given Data:

Process	Burst Time (ms)
P1	24
P2	3
P3	3

(a) Gantt Chart — FCFS Order: P1 → P2 → P3

Gantt Chart – FCFS Scheduling (Q1)



Average Waiting Time = 17 ms

(b) Waiting Times:

Since all processes arrive at time 0, Waiting Time = Start Time – Arrival Time = Start Time.

Process	Burst Time	Start Time	Waiting Time
P1	24	0	$0 - 0 = 0$ ms
P2	3	24	$24 - 0 = 24$ ms
P3	3	27	$27 - 0 = 27$ ms

(c) Average Waiting Time:**Key Formulas**

Average Waiting Time = $(0 + 24 + 27) / 3 = 51 / 3 = 17$ milliseconds

Question 2 — FCFS With Improved Arrival Order

Question

Given:

- Three processes: P1, P2, P3
- Arrival time of all processes = 0
- Order of arrival: P2 → P3 → P1
- CPU burst times:
 - P1 = 24 ms
 - P2 = 3 ms
 - P3 = 3 ms

Using: First-Come, First-Served (FCFS) scheduling algorithm

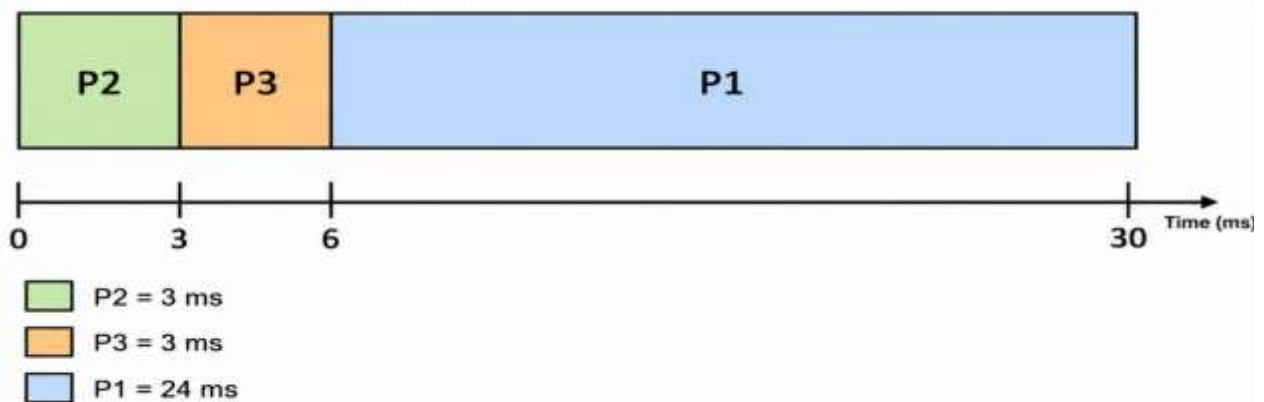
Find:

1. Draw the Gantt chart.
2. Calculate the waiting time for each process.
3. Calculate the average waiting time.
4. Compare the result with Question 1 and explain the difference.

Answer

(a) Gantt Chart — FCFS Order: P2 → P3 → P1

Gantt Chart – FCFS Scheduling (Q2: P2→P3→P1)



Average Waiting Time = 3 ms (vs 17 ms in Q1 — shows Convoy Effect impact)

(b) Waiting Times:

Process	Burst Time	Start Time	Waiting Time
P2	3	0	0 ms
P3	3	3	3 ms
P1	24	6	6 ms

(c) Average Waiting Time:

Key Formulas

Average Waiting Time = $(0 + 3 + 6) / 3 = 9 / 3 = 3$ milliseconds

(d) Comparison:

Note: Question 1 order (P1→P2→P3) gave Average Wait = 17 ms. This order (P2→P3→P1) gives 3 ms. The same processes in a different arrival order yield dramatically different results. This illustrates the Convoy Effect in FCFS.

Question 3 — FCFS With Different Arrival Times

Question

Given:

- Five processes with arrival and burst times:
 - P1: Burst = 6, Arrival = 2
 - P2: Burst = 2, Arrival = 5
 - P3: Burst = 8, Arrival = 1
 - P4: Burst = 3, Arrival = 0
 - P5: Burst = 4, Arrival = 4

Using: First-Come, First-Served (FCFS) scheduling algorithm (based on arrival time)

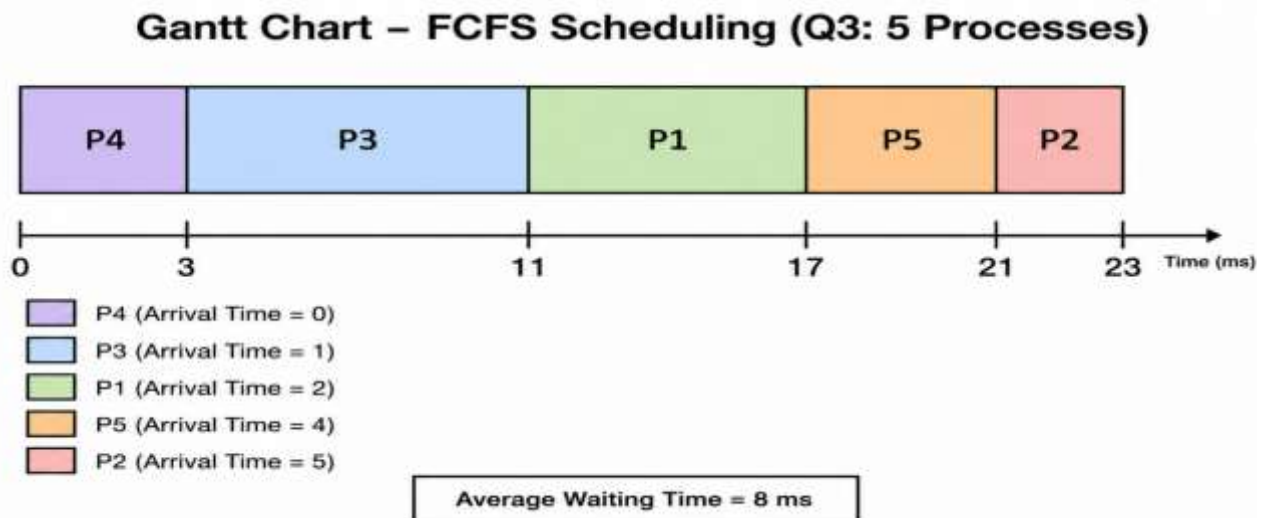
Find:

- Draw the Gantt chart.
- Calculate the waiting time for each process (Waiting Time = Start Time - Arrival Time).
- Calculate the average waiting time.

Process	Burst Time (ms)	Arrival Time (ms)
P1	6	2
P2	2	5
P3	8	1
P4	3	0
P5	4	4

Execution Order (sorted by arrival): P4(t=0) → P3(t=1) → P1(t=2) → P5(t=4) → P2(t=5)

(a) Gantt Chart:



(b) Waiting Times (Waiting = Start – Arrival):

Process	Arrival	Burst	Start Time	Waiting Time	Turnaround Time
P4	0	3	0	$0 - 0 = 0$	3
P3	1	8	3	$3 - 1 = 2$	10
P1	2	6	11	$11 - 2 = 9$	15
P5	4	4	17	$17 - 4 = 13$	17
P2	5	2	21	$21 - 5 = 16$	18

(c) Average Waiting Time:

Key Formulas

Average Waiting Time = $(0 + 2 + 9 + 13 + 16) / 5 = 40 / 5 = 8$ milliseconds

FCFS — Convoy Effect

The convoy effect is a major problem with FCFS. When a long process arrives first, all short processes behind it must wait a very long time.

Imagine one big truck on a narrow road — all the small cars behind it are stuck. This is exactly what happens to short processes waiting behind a long process in FCFS.

FCFS — Advantages and Disadvantages

Advantages	Disadvantages
Very simple to understand and implement	Convoy Effect: short jobs wait behind long ones
Fair in order of arrival	High average waiting time
No starvation — every process eventually runs	Not suitable for time-sharing systems
Low overhead	Cannot prioritize urgent processes

7.2 Shortest-Job-First (SJF) Scheduling

SJF scheduling selects the process with the shortest next CPU burst time. The intuition is simple: get small jobs out of the way quickly.

- Optimal: SJF gives the minimum possible average waiting time for any set of processes.
- Requires knowledge of the next CPU burst length — in practice, this is estimated.

Variant	Description
Non-preemptive SJF	Once the CPU is given to a process, it runs until its burst completes. The shortest among arrived processes is chosen next.
Preemptive SJF (SRTF)	Shortest Remaining Time First. If a new process arrives with a shorter remaining time than the current process, the current process is preempted.

Question 4 — SJF Without Arrival Time (Non-Preemptive)

Question

Given:

- Four processes (All arrive at time 0)
- CPU Burst Times:
 - P1 = 6 ms
 - P2 = 8 ms
 - P3 = 7 ms
 - P4 = 3 ms

Using: Non-Preemptive Shortest Job First (SJF) Scheduling

Find:

1. Determine the execution order of processes.
2. Draw the Gantt chart.
3. Calculate the waiting time for each process.
4. Calculate the average waiting time.

Answer

Given Data:

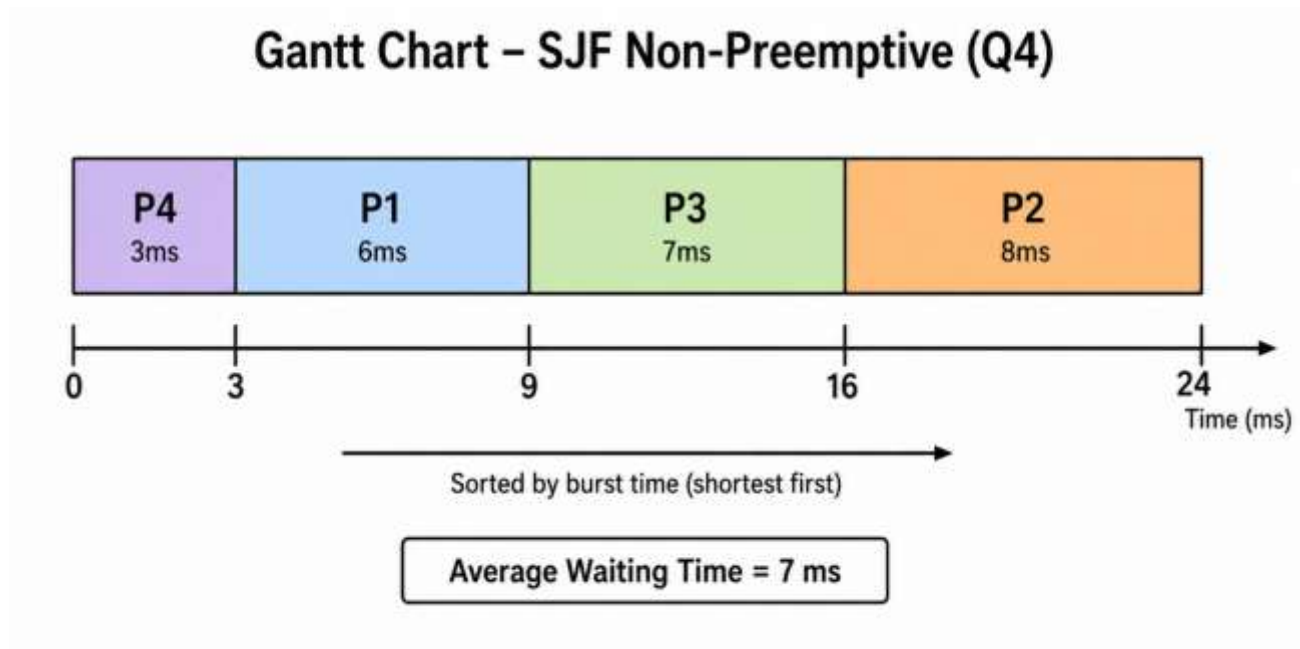
Process	Burst Time (ms)
P1	6
P2	8

P3	7
P4	3

(a) Execution Order: Sort by burst time (shortest first)

P4 (3ms) → P1 (6ms) → P3 (7ms) → P2 (8ms)

(b) Gantt Chart:



(c) Waiting Times (all arrive at t=0, so Wait = Start Time):

Process	Burst	Start Time	Waiting Time
P4	3	0	0 ms
P1	6	3	3 ms
P3	7	9	9 ms
P2	8	16	16 ms

(d) Average Waiting Time:

Key Formulas

Average Waiting Time = $(0 + 3 + 9 + 16) / 4 = 28 / 4 = 7$ milliseconds

Question 5 — SJF With Arrival Time (Non-Preemptive)

Question

Given:

- Four processes with arrival and burst times:
 - P1: Arrival = 0, Burst = 7
 - P2: Arrival = 2, Burst = 4
 - P3: Arrival = 4, Burst = 1
 - P4: Arrival = 5, Burst = 4

Using: Non-Preemptive Shortest Job First (SJF) Scheduling

Find:

1. Trace the execution step-by-step.
2. Draw the Gantt chart.
3. Calculate waiting time and turnaround time for each process.
4. Calculate the average waiting time.

Answer

Given Data:

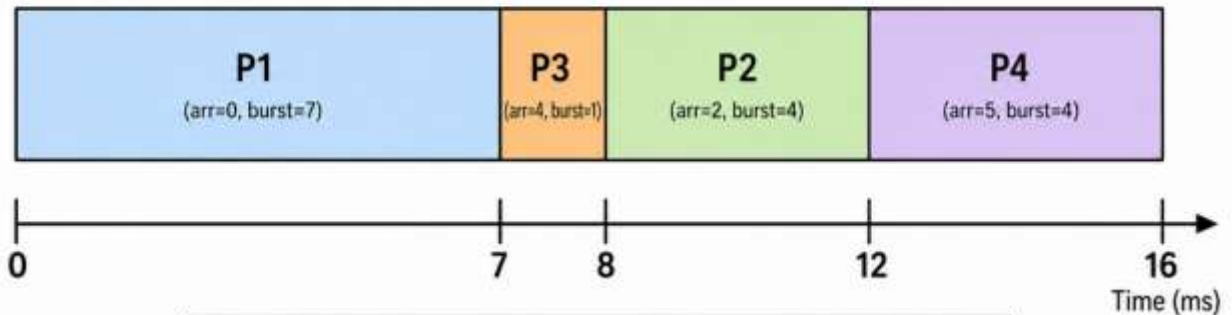
Process	Arrival Time	Burst Time
P1	0.0	7
P2	2.0	4
P3	4.0	1
P4	5.0	4

(a) Step-by-Step Execution Trace:

1. $t=0$: Only P1 has arrived. Start P1 (burst=7). P1 runs: $0 \rightarrow 7$.
2. $t=7$: P1 finishes. Available: P2(4), P3(1), P4(4). Shortest = P3(1). Start P3: $7 \rightarrow 8$.
3. $t=8$: P3 finishes. Available: P2(4), P4(4). Tie $\rightarrow$ pick P2. P2 runs: $8 \rightarrow 12$.
4. $t=12$: P2 finishes. Only P4 remains. P4 runs: $12 \rightarrow 16$.

(b) Gantt Chart:

Gantt Chart – SJF Non-Preemptive (Q5: With Arrival Times)



At t=7, available: P2(4), P3(1), P4(4) → P3 chosen (shortest burst)

Average Waiting Time = 4 ms

(c) Waiting and Turnaround Times:

Process	Arrival	Burst	Start	Waiting (Start-Arrival)	Turnaround
P1	0	7	0	$0 - 0 = 0$	7
P2	2	4	8	$8 - 2 = 6$	10
P3	4	1	7	$7 - 4 = 3$	4
P4	5	4	12	$12 - 5 = 7$	11

(d) Average Waiting Time:

Key Formulas

Average Waiting Time = $(0 + 6 + 3 + 7) / 4 = 16 / 4 = 4$ milliseconds

Question 6 — Preemptive SJF / SRTF With Arrival Time

Question

Given:

- Four processes with arrival and burst times:
 - P1: Arrival = 0.0, Burst = 7
 - P2: Arrival = 2.0, Burst = 4
 - P3: Arrival = 4.0, Burst = 1
 - P4: Arrival = 5.0, Burst = 4

Using: Preemptive Shortest Job First (SJF) / Shortest Remaining Time First (SRTF) Scheduling

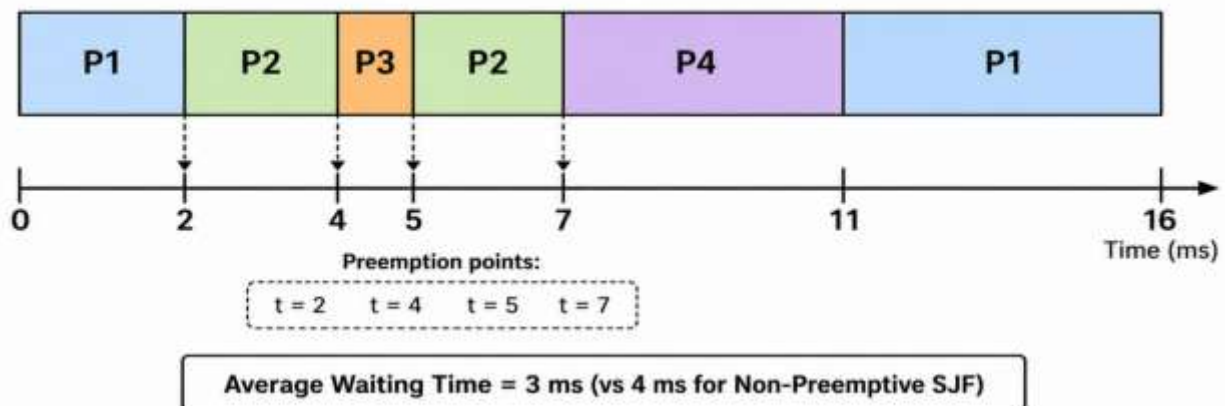
Find:

- Trace the execution step-by-step, showing all preemptions.
- Draw the Gantt chart.
- Calculate the waiting time for each process.
- Calculate the average waiting time and compare it with Non-Preemptive SJF.

(a) Step-by-Step Execution Trace (SRTF):

- $t=0$: P1 arrives (remaining=7). Only process $\rightarrow$ Start P1.
- $t=2$: P2 arrives (burst=4). P1 has remaining=5. Since $4 < 5 \rightarrow$ PREEMPT P1. Start P2.
- $t=4$: P3 arrives (burst=1). P2 has remaining=2. Since $1 < 2 \rightarrow$ PREEMPT P2. Start P3.
- $t=5$: P3 finishes (burst=1 done). P4 arrives (burst=4). Compare: P1(rem=5), P2(rem=2), P4(4). Shortest=P2(2) $\rightarrow$ Start P2.
- $t=7$: P2 finishes. Compare: P1(rem=5), P4(4). Shortest=P4(4) $\rightarrow$ Start P4.
- $t=11$: P4 finishes. Only P1 remains (rem=5) $\rightarrow$ Start P1. Runs: 11 $\rightarrow$ 16.

(b) Gantt Chart (SRTF):



(c) Waiting Times:

Process	Arrival	Burst	Waiting Time	Calculation
P1	0	7	9 ms	(0→2 ran 2ms) + (11→16 ran 5ms) = 7ms used; wait = 16-7-0 = 9
P2	2	4	1 ms	Ran at 2→4 (2ms), 5→7 (2ms); wait = 5-4 = 1
P3	4	1	0 ms	Arrived t=4, started immediately; wait = 4-4 = 0
P4	5	4	2 ms	Arrived t=5, started t=7; wait = 7-5 = 2

(d) Average Waiting Time:

Key Formulas

Average Waiting Time = $(9 + 1 + 0 + 2) / 4 = 12 / 4 = 3$ milliseconds

Comparison: Non-preemptive SJF (Q5) = 4 ms | Preemptive SRTF (Q6) = 3 ms
SRTF achieves lower average waiting time by always picking the shortest remaining job.

SJF — Why is it Optimal?

SJF produces the minimum average waiting time because it always reduces the waiting time of the most processes. By completing short jobs first, fewer processes accumulate time in the queue.

SJF — Advantages and Disadvantages

Advantages	Disadvantages
Optimal — minimum average waiting time	Requires advance knowledge of burst time
Short jobs receive fast service	Long jobs may starve (wait indefinitely)
Reduces convoy effect vs FCFS	Burst time prediction can be inaccurate
SRTF variant is very responsive	More complex to implement than FCFS

7.3 Priority Scheduling

Priority scheduling assigns a priority number to each process. The CPU is given to the process with the highest priority.

- Convention: smaller priority number = higher priority (priority 1 is higher than priority 5).
- Can be preemptive or non-preemptive.
- SJF is a special case of priority scheduling where $\text{priority} = 1 / \text{burst time}$.

The Starvation Problem

Starvation occurs when a low-priority process never gets the CPU because higher-priority processes keep arriving. It can wait in the ready queue indefinitely.

Solution: Aging

Aging gradually increases the priority of a process the longer it waits. This ensures that even the lowest-priority process will eventually get the CPU.

Question 7 — Priority Scheduling (Non-Preemptive)

Question

Given:

- Seven processes with Priority, Arrival, and Burst times
(Note: Smaller priority number = Higher priority)
 - P1: Priority = 2, Arrival = 0, Burst = 3
 - P2: Priority = 6, Arrival = 2, Burst = 5
 - P3: Priority = 3, Arrival = 1, Burst = 4
 - P4: Priority = 5, Arrival = 4, Burst = 2
 - P5: Priority = 7, Arrival = 6, Burst = 9
 - P6: Priority = 4, Arrival = 5, Burst = 4
 - P7: Priority = 10, Arrival = 7, Burst = 10

Using: Non-Preemptive Priority Scheduling

Find:

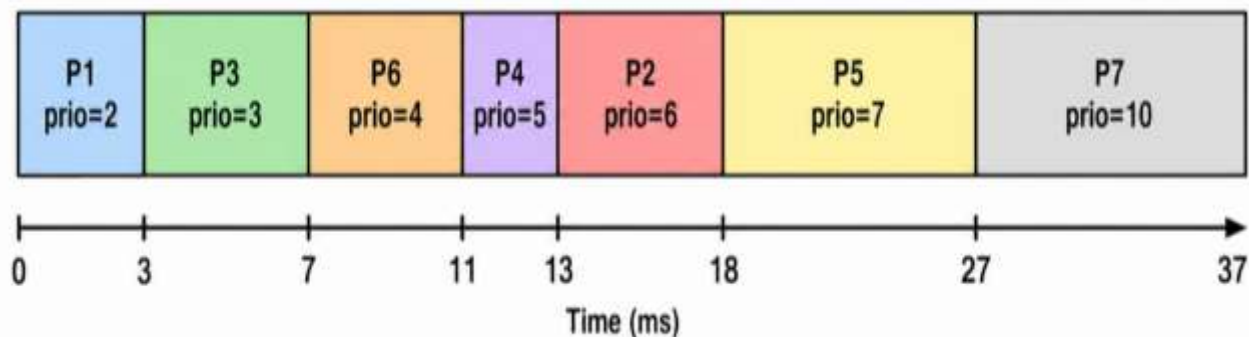
1. Determine the execution order of processes.
2. Draw the Gantt chart.
3. Calculate the following for each process:
 - Completion Time (CT)
 - Turnaround Time (TAT)
 - Waiting Time (WT)
 - Response Time (RT)
4. Calculate the average waiting time.

Given Data:

Process	Priority	Arrival Time	Burst Time
P1	2 (highest)	0	3
P2	6	2	5
P3	3	1	4
P4	5	4	2
P5	7	6	9
P6	4	5	4
P7	10 (lowest)	7	10

(a) Execution Order Trace:

1. $t=0$: Only P1 arrived (priority 2). Run P1: $0 \rightarrow 3$.
2. $t=3$: P1 done. Available: P3(prio=3). Run P3: $3 \rightarrow 7$.
3. $t=7$: P3 done. Available: P2(6), P4(5), P5(7), P6(4), P7(10). Highest priority = P6(4). Run P6: $7 \rightarrow 11$.
4. $t=11$: P6 done. Remaining: P2(6), P4(5), P5(7), P7(10). Highest = P4(5). Run P4: $11 \rightarrow 13$.
5. $t=13$: P4 done. Remaining: P2(6), P5(7), P7(10). Highest = P2(6). Run P2: $13 \rightarrow 18$.
6. $t=18$: P2 done. Remaining: P5(7), P7(10). Highest = P5(7). Run P5: $18 \rightarrow 27$.
7. $t=27$: P5 done. Only P7 left. Run P7: $27 \rightarrow 37$.

(b) Gantt Chart:

Lower priority number = Higher priority.
Average Waiting Time ≈ 7.43 ms

(c) Completion, Turnaround, Waiting, and Response Times:

Formulas: Turnaround = Completion – Arrival | Waiting = Turnaround – Burst | Response = Start – Arrival

Process	Priority	Arrival	Burst	Completion	Turnaround	Waiting	Response
P1	2	0	3	3	3	0	0
P2	6	2	5	18	16	11	13 (waited until t=13 to start)
P3	3	1	4	7	6	2	3
P4	5	4	2	13	9	7	11 (waited until t=11 to start)
P5	7	6	9	27	21	12	18
P6	4	5	4	11	6	2	7
P7	10	7	10	37	30	18	27

(d) Average Waiting Time:

Key Formulas

Average Waiting Time = $(0 + 11 + 2 + 7 + 12 + 2 + 18) / 7 = 52 / 7 \approx 7.43$ milliseconds

Question 8 — Priority Scheduling (Preemptive)

Question

Given:

- Five processes with Priority, Burst, and Arrival times
(Note: Smaller number = Higher priority)
 - P1: Priority = 1, Burst = 4, Arrival = 0
 - P2: Priority = 2, Burst = 3, Arrival = 0
 - P3: Priority = 1, Burst = 7, Arrival = 6
 - P4: Priority = 3, Burst = 4, Arrival = 11
 - P5: Priority = 2, Burst = 2, Arrival = 12

Using: Preemptive Priority Scheduling

Find:

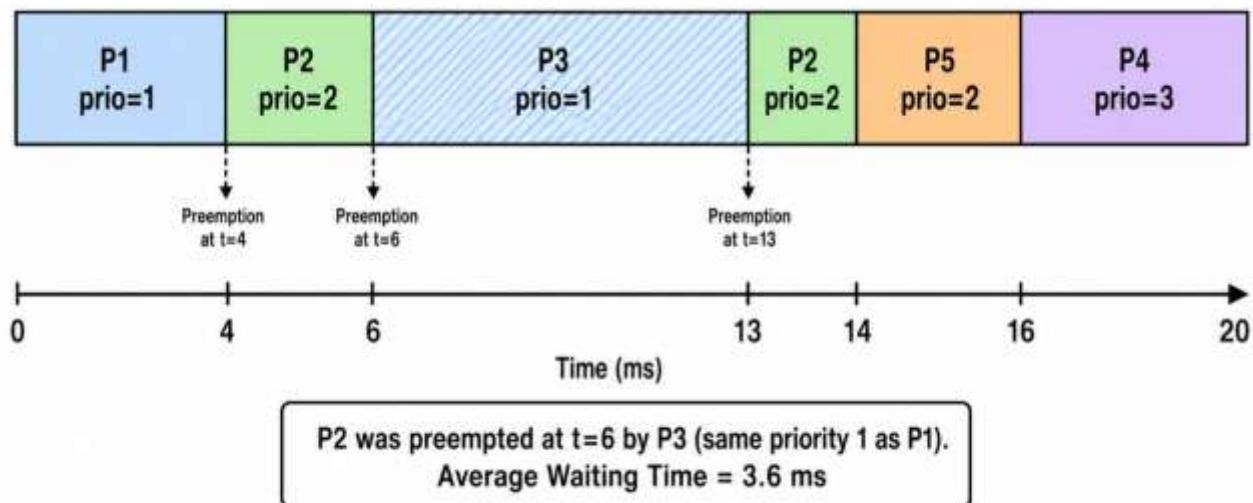
- Trace the execution step-by-step, showing all preemptions.
- Draw the Gantt chart.
- Calculate the waiting time for each process.
- Calculate the average waiting time.

Given Data:

Process	Priority	Burst Time	Arrival Time
P1	1 (highest)	4	0
P2	2	3	0
P3	1 (highest)	7	6
P4	3	4	11
P5	2	2	12

(a) Step-by-Step Trace:

1. $t=0$: P1 (prio=1) and P2 (prio=2) arrive. Highest = P1. Start P1: $0 \rightarrow 4$.
2. $t=4$: P1 finishes. Only P2 available (prio=2). Start P2: $4 \rightarrow 6$.
3. $t=6$: P3 arrives (prio=1 — same as P1, higher than P2's prio=2). PREEMPT P2. Start P3: $6 \rightarrow 13$.
4. $t=11$: P4 arrives (prio=3). P3 (prio=1) still higher. P3 continues.
5. $t=12$: P5 arrives (prio=2). P3 (prio=1) still higher. P3 continues until 13.
6. $t=13$: P3 finishes. Remaining: P2(1ms left, prio=2), P4(prio=3), P5(prio=2). Highest prio=2 $\rightarrow$ P2 or P5. Run P2: $13 \rightarrow 14$.
7. $t=14$: P2 finishes. Remaining: P4(prio=3), P5(prio=2). Higher = P5. Run P5: $14 \rightarrow 16$.
8. $t=16$: P5 finishes. Only P4 remains. Run P4: $16 \rightarrow 20$.

(b) Gantt Chart:**(c) Waiting Times:**

For preemptive scheduling: Waiting = Total time spent in ready queue = Turnaround – Burst.

Process	Arrival	Burst	Waiting Time	Calculation
P1	0	4	0 ms	Started at $t=0$, no wait: $0 - 0 = 0$
P2	0	3	11 ms	First start=4, preempted at 6, resumed 13: $(4-0) + (13-6) = 4+7 = 11$

P3	6	7	0 ms	Arrived t=6, started immediately: $6 - 6 = 0$
P4	11	4	5 ms	Arrived t=11, started t=16: $16 - 11 = 5$
P5	12	2	2 ms	Arrived t=12, started t=14: $14 - 12 = 2$

(d) Average Waiting Time:

Key Formulas

Average Waiting Time = $(0 + 11 + 0 + 5 + 2) / 5 = 18 / 5 = 3.6$ milliseconds

Priority Scheduling — Advantages and Disadvantages

Advantages	Disadvantages
Important/urgent processes get CPU first	Starvation: low-priority may never run
Flexible — priorities can be assigned by policy	Choosing the right priorities is difficult
Can be used for real-time systems	Aging adds implementation complexity
Works for both preemptive and non-preemptive	Priority inversion can cause bugs

7.4 Round Robin (RR) Scheduling

Round Robin is designed specifically for time-sharing systems. It gives each process a small, equal time slice called the time quantum (q). After the quantum expires, the process is preempted and moved to the back of the ready queue.

- Preemptive — the CPU is forcibly taken after each quantum.
- Fair — every process gets an equal share of the CPU.
- No starvation — every process gets a turn within $(n - 1) \times q$ time units.
- The time quantum is typically 10–100 milliseconds.

Maximum Wait Formula

With n processes and time quantum q :

$$\text{Maximum waiting time} = (n - 1) \times q$$

Effect of Time Quantum Size

Time Quantum	Behavior	Pros	Cons
Very Large (e.g., ∞)	Behaves like FCFS — process runs to completion	Low context switch overhead	Poor responsiveness
Large (e.g., $>$ burst)	Similar to FCFS	Less context switching	Longer response time
Optimal (10–100 ms)	True Round Robin behavior	Good balance	Some overhead
Very Small (e.g., 1 ms)	Process rarely finishes in one turn	Excellent responsiveness	Huge overhead from context switching

Note: The time quantum should be large compared to context switch time. Context switch typically takes < 10 microseconds, so a quantum of 10–100 ms gives $< 1\%$ overhead.

Question 9 — Round Robin (Time Quantum $q = 4$)

Question

Given:

- Three processes (all arrive at time 0)
- CPU Burst Times:
 - P1 = 24 ms
 - P2 = 3 ms
 - P3 = 3 ms
- Time Quantum: $q = 4$

Using: Round Robin (RR) Scheduling

Find:

1. Trace the execution step-by-step.
2. Draw the Gantt chart.
3. Calculate the waiting time for each process.
4. Calculate the average waiting time.

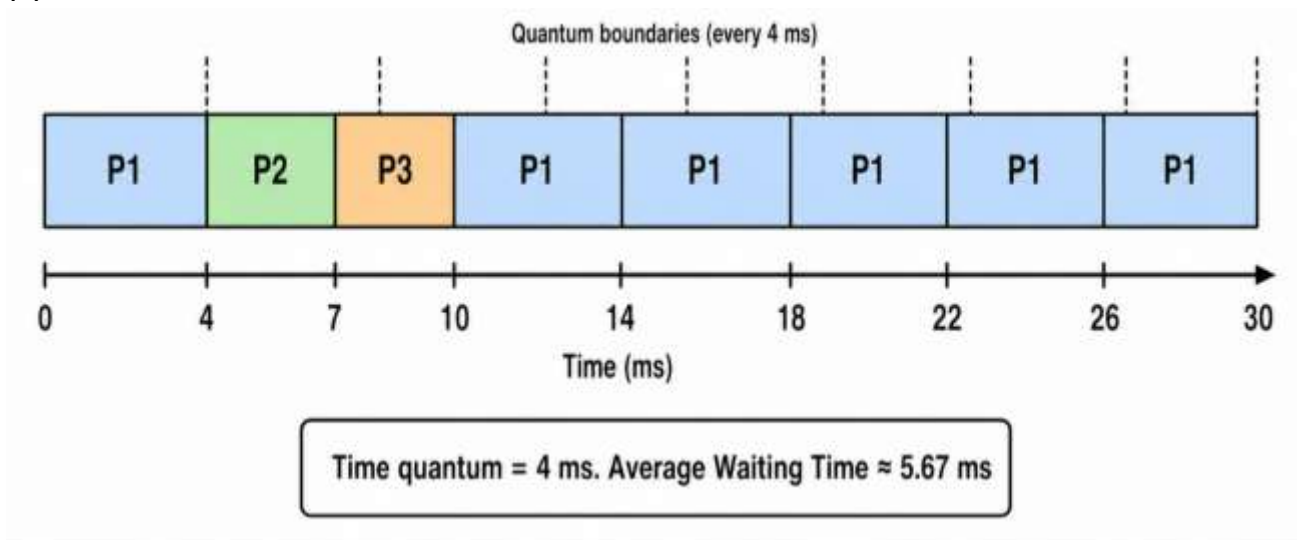
Given Data:

Process	Burst Time (ms)
P1	24
P2	3
P3	3

(a) Step-by-Step Trace ($q=4$):

1. $t=0-4$: P1 runs 4ms. P1 remaining = 20ms. P1 re-queued.
2. $t=4-7$: P2 runs 3ms (finishes — burst < quantum). P2 done.
3. $t=7-10$: P3 runs 3ms (finishes — burst < quantum). P3 done.
4. $t=10-14$: P1 runs 4ms. P1 remaining = 16ms.
5. $t=14-18$: P1 runs 4ms. P1 remaining = 12ms.
6. $t=18-22$: P1 runs 4ms. P1 remaining = 8ms.
7. $t=22-26$: P1 runs 4ms. P1 remaining = 4ms.
8. $t=26-30$: P1 runs final 4ms. P1 done.

(b) Gantt Chart:



(c) Waiting Times:

For RR, waiting time = time the process spent waiting (not running) in the ready queue.

Process	Burst	Completion	Waiting Time	Calculation
P1	24	30	6 ms	Ran 0→4 (4ms), then waited 4→10, then ran 10 onward. First wait = 10-4 = 6ms
P2	3	7	4 ms	Arrived at t=0, first run at t=4: Wait = 4 ms
P3	3	10	7 ms	Arrived at t=0, first run at t=7: Wait = 7 ms

(d) Average Waiting Time:

Key Formulas

Average Waiting Time = $(6 + 4 + 7) / 3 = 17 / 3 \approx 5.67$ milliseconds

Question 10 — Round Robin (Time Quantum $q = 2$)

Question

Given:

- Three processes (all arrive at time 0)
- CPU Burst Times:
 - P1 = 4 ms
 - P2 = 3 ms
 - P3 = 5 ms
- Initial Queue Order: P1 → P2 → P3
- Time Quantum: $q = 2$

Using: Round Robin (RR) Scheduling

Find:

1. Trace the execution step-by-step.
2. Draw the Gantt chart.
3. Calculate the waiting time for each process.
4. Calculate the average waiting time.

Answer

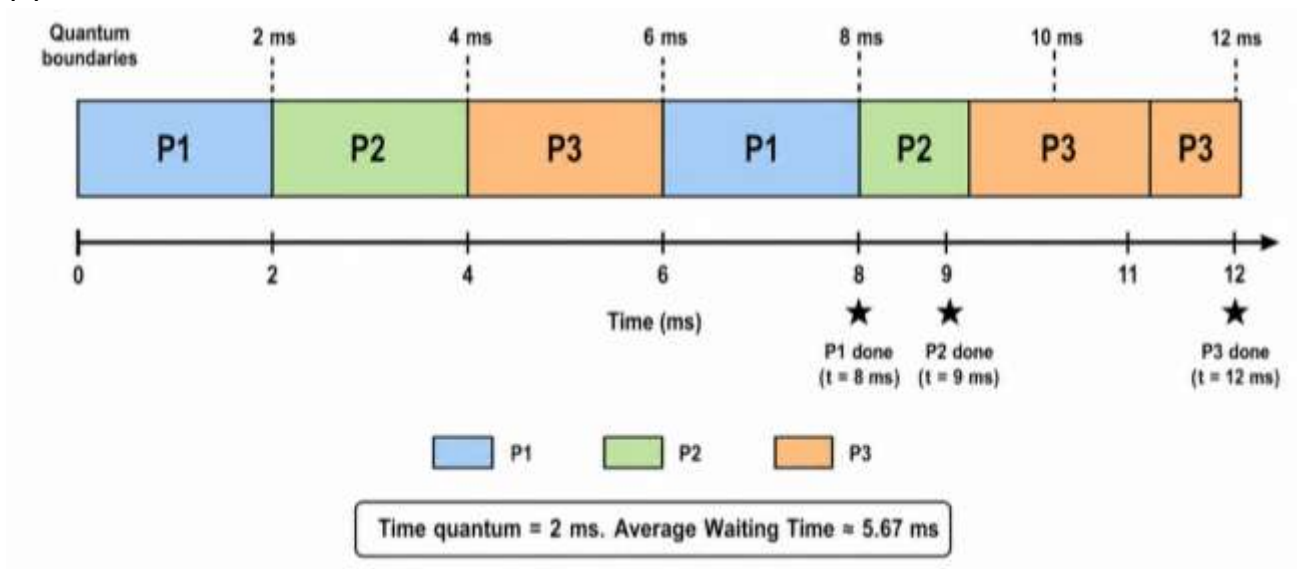
Given Data:

Process	Burst Time (ms)
P1	4
P2	3
P3	5

(a) Step-by-Step Trace ($q=2$):

1. $t=0-2$: P1 runs 2ms. P1 remaining = 2ms. Re-queue.
2. $t=2-4$: P2 runs 2ms. P2 remaining = 1ms. Re-queue.
3. $t=4-6$: P3 runs 2ms. P3 remaining = 3ms. Re-queue.
4. $t=6-8$: P1 runs 2ms. P1 remaining = 0ms. P1 DONE.
5. $t=8-9$: P2 runs 1ms (finishes). P2 DONE.
6. $t=9-11$: P3 runs 2ms. P3 remaining = 1ms. Re-queue.
7. $t=11-12$: P3 runs 1ms (finishes). P3 DONE.

(b) Gantt Chart:



(c) Waiting Times:

Process	Burst	Completion	Waiting Time	Calculation
P1	4	8	4 ms	Waited: $0 + (6-2) = 0 + 4 = 4\text{ms}$
P2	3	9	6 ms	Waited: $2 + (8-4) = 2 + 4 = 6\text{ms}$
P3	5	12	7 ms	Waited: $4 + (9-6) = 4 + 3 = 7\text{ms}$

(d) Average Waiting Time:

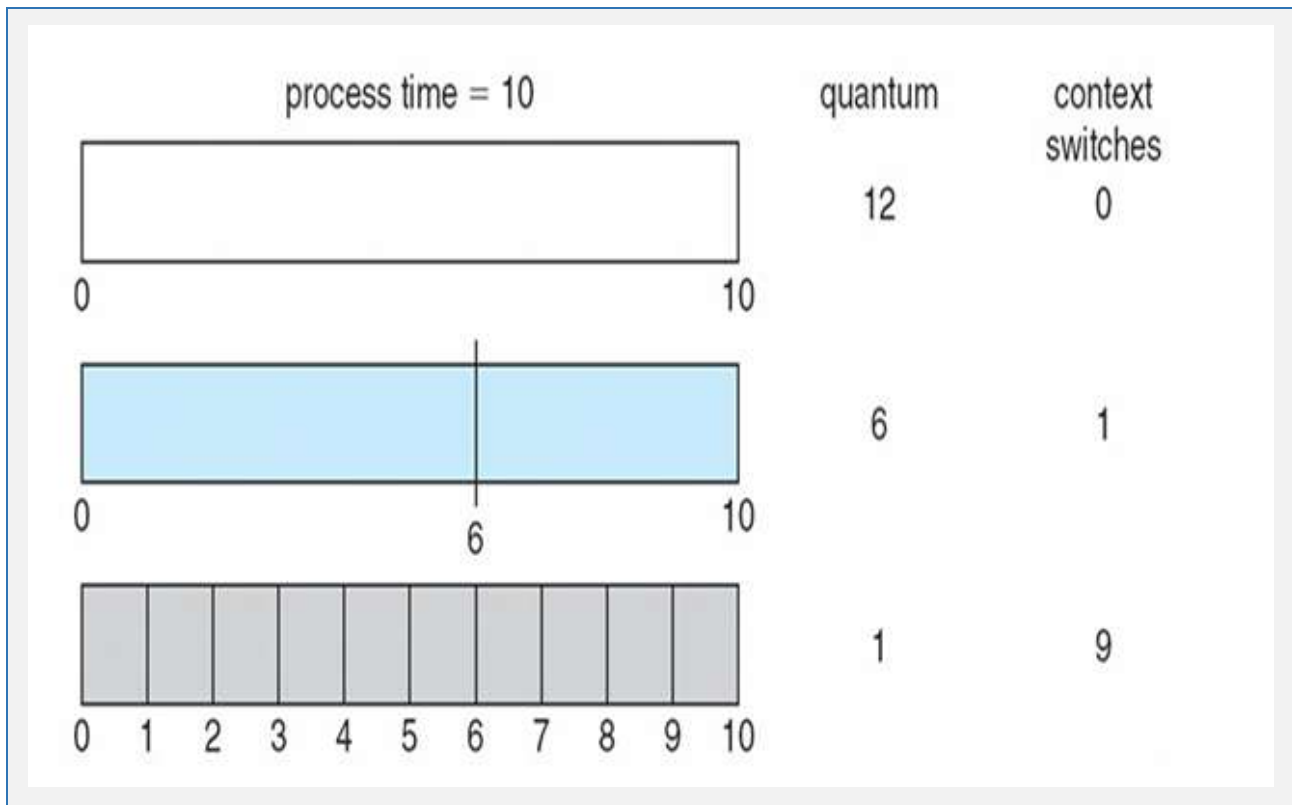
Key Formulas

Average Waiting Time = $(4 + 6 + 7) / 3 = 17 / 3 \approx 5.67$ milliseconds

Time Quantum and Context Switch Overhead

Time Quantum	Number of Context Switches	Observation
12 (> burst of 10)	0	Process finishes in one turn; no context switch needed
6	1	Process is split into 2 parts; 1 context switch
1	9	Process split into 10 parts; 9 context switches — very high overhead

Time Quantum and Context Switch Overhead



Time Quantum	Number of Context Switches	Observation
12 (> burst of 10)	0	Process finishes in one turn; no context switch needed
6	1	Process is split into 2 parts; 1 context switch
1	9	Process split into 10 parts; 9 context switches — very high overhead

Round Robin — Advantages and Disadvantages

Advantages	Disadvantages
Fair — all processes get equal CPU time	Higher average turnaround time than SJF
No starvation	Too small a quantum causes excessive context switching
Good response time for short processes	CPU-bound processes suffer from frequent preemptions

Advantages	Disadvantages
Ideal for time-sharing/interactive systems	Performance depends heavily on choosing the right quantum



Chapter Summary

- Round Robin gives each process a time quantum (slice) of CPU time.
- After the quantum, the process is preempted and re-queued.
- No starvation — every process gets a turn within $(n-1) \times q$ time.
- Smaller quantum = better response but more context switch overhead.
- Larger quantum = less overhead but behaves like FCFS.
- Optimal quantum is usually 10–100 milliseconds.

Key Formulas

Maximum Wait Time = $(n - 1) \times q$

Turnaround Time = Completion Time - Arrival Time

Waiting Time = Turnaround Time - Burst Time

Avg Waiting Time = Sum of All Waiting Times $\div$ Number of Processes

Appendix: Algorithm Comparison Summary

Algorithm	Type	Starvation?	Preemptive?	Best For	Avg Wait
FCFS	Non-preemptive	No	No	Simple batch systems	High
SJF (non-preemptive)	Non-preemptive	Yes (long jobs)	No	Batch with known bursts	Optimal
SRTF (preemptive SJF)	Preemptive	Yes	Yes	Minimize avg waiting	Optimal
Priority (non-preemptive)	Non-preemptive	Yes	No	Prioritized batch	Medium
Priority (preemptive)	Preemptive	Yes (use aging)	Yes	Real-time systems	Low
Round Robin	Preemptive	No	Yes	Time-sharing, interactive	Medium